

Assignment 2: Styling and Navigation for Fitness Tracker App

Release Date: September 28, 2025

Due Date: Sunday, October 12, 2025, 11:59 PM EST

Weight: 12.5% of final grade

Objective

Extend the basic fitness tracker app from Assignment 1 by incorporating styling with `StyleSheet` and `Flexbox`, and adding multi-screen navigation using `React Navigation`. This assignment focuses on improving the user interface with consistent styling (e.g., colors, padding, and card-like layouts) and implementing navigation between screens (e.g., a Home screen for the activity list, an Add/Edit Activity screen for logging or updating activities, and an Activity Details screen for viewing individual activities). This builds foundational skills in mobile app design, layout, and navigation in React Native.

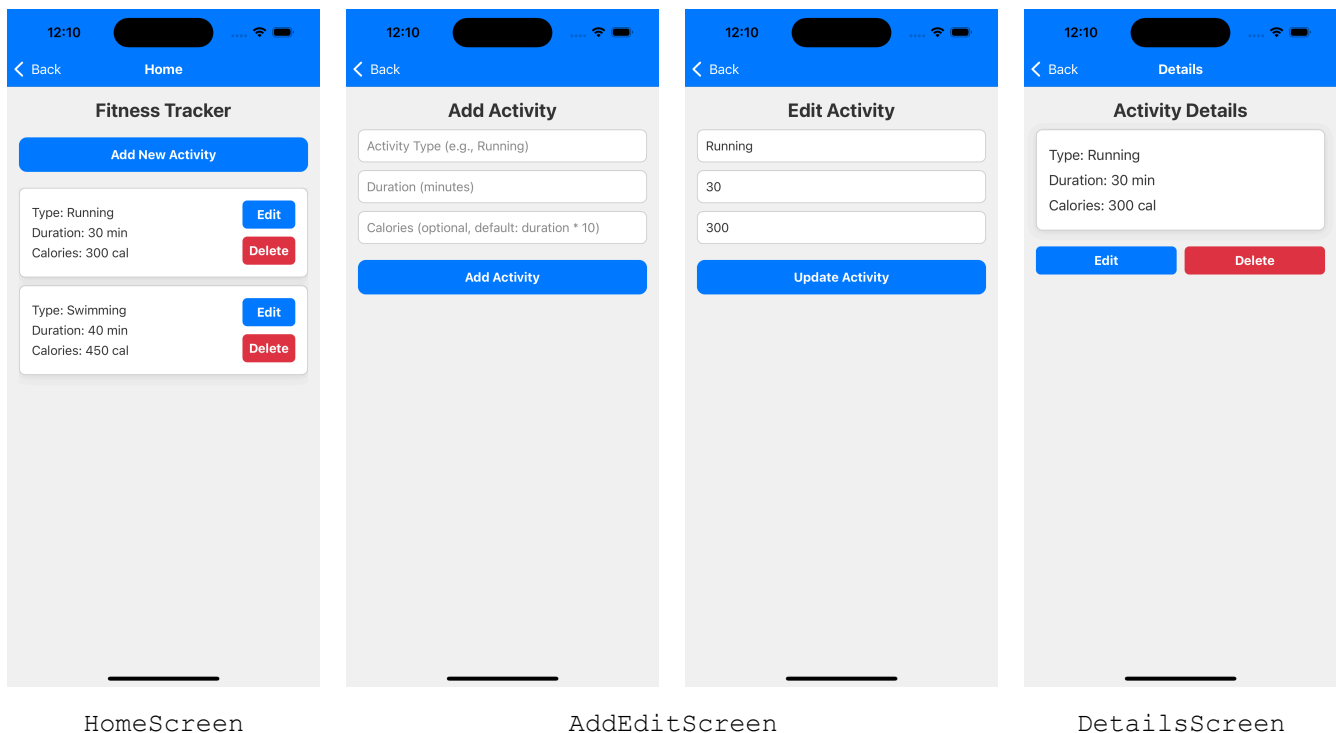
Technologies

- **React Native:** For building the mobile app UI and functionality.
- **TypeScript:** For type-safe development of the app.
- **Expo CLI:** For project setup, development, and testing via Expo Go or emulators.
- **React Navigation:** For implementing multi-screen navigation.
- **Node.js (v22):** Required for Expo CLI and project dependencies.

Assignment Requirements

This is a three-screen app built with `React Navigation`'s stack navigator. The screens are:

- `screens/HomeScreen.tsx` : The main home screen that displays a list of activities using a `FlatList`, along with a button to add a new activity.
- `screens/AddEditScreen.tsx` : A form screen for adding a new activity or editing an existing one. It reuses the form logic from Assignment 1, with input validation.
- `screens/DetailsScreen.tsx` : A details screen that shows information about a selected activity, with options to edit or delete it.



HomeScreen

AddEditScreen

DetailsScreen

Build upon your Assignment 1 submission to add styling and navigation. All functionality from Assignment 1 (adding, updating, deleting, displaying activities, and input validation) must remain intact. The app continues to manage activities in memory using `useState` and does not require persistent storage or deployment.

1 React Native App Setup

- **Starter Code:** Use your completed Assignment 1 project as the starting point. Download the reference starter project by [clicking this link](#) to obtain `assignment-2.tar.gz`. This archive includes an updated `package.json` and `package-lock.json` (with React Navigation dependencies: `@react-navigation/native`, and `@react-navigation/native-stack`), and additional starter files like `components/Card.tsx` and `constants/colors.ts`. Extract the `.tar.gz` file, navigate to the project directory, run `npm install` to install dependencies, and edit the necessary files to implement the required functionality.
- **Storage and Types:** Keep the in-memory array managed by `useState`, and the `Activity` type (defined in `types.ts`) from Assignment 1. Extend `types.ts` with navigation types for type safety:

1. Define a `RootStackParamList` type mapping screen names to route parameters:

- `"Home"` → no params
- `"AddEdit"` → optional `Activity`
- `"Details"` → required `Activity`

2. Create a `NavigationProp` type alias using `NativeStackNavigationProp<RootStackParamList>`.

Import these types in your `screens/` and `components/` for props and `useNavigation`. Lift the `activities` state to `App.tsx` and pass the `activities` array and `setActivities` function as props to screens that need them.

- **Constants and Global Styles:** The starter code provides `constants/colors.ts` with the app's color scheme. You need to create a `styles/globalStyles.ts` file with the following **exact styles**:

1. `container`

- `flex: 1`
- `padding: 15`
- `backgroundColor: colors.background`

2. `headerText`

- `fontSize: 24`
- `fontWeight: "bold"`
- `color: colors.textPrimary`
- `textAlign: "center"`
- `marginBottom: 10`

Use `StyleSheet.create` to define them and import these styles in your screens.

2 Custom Components

Create the following custom components in the `components/` directory. These promote code reuse and consistency across screens. The starter code provides `Card.tsx` as a base for card-like layouts.

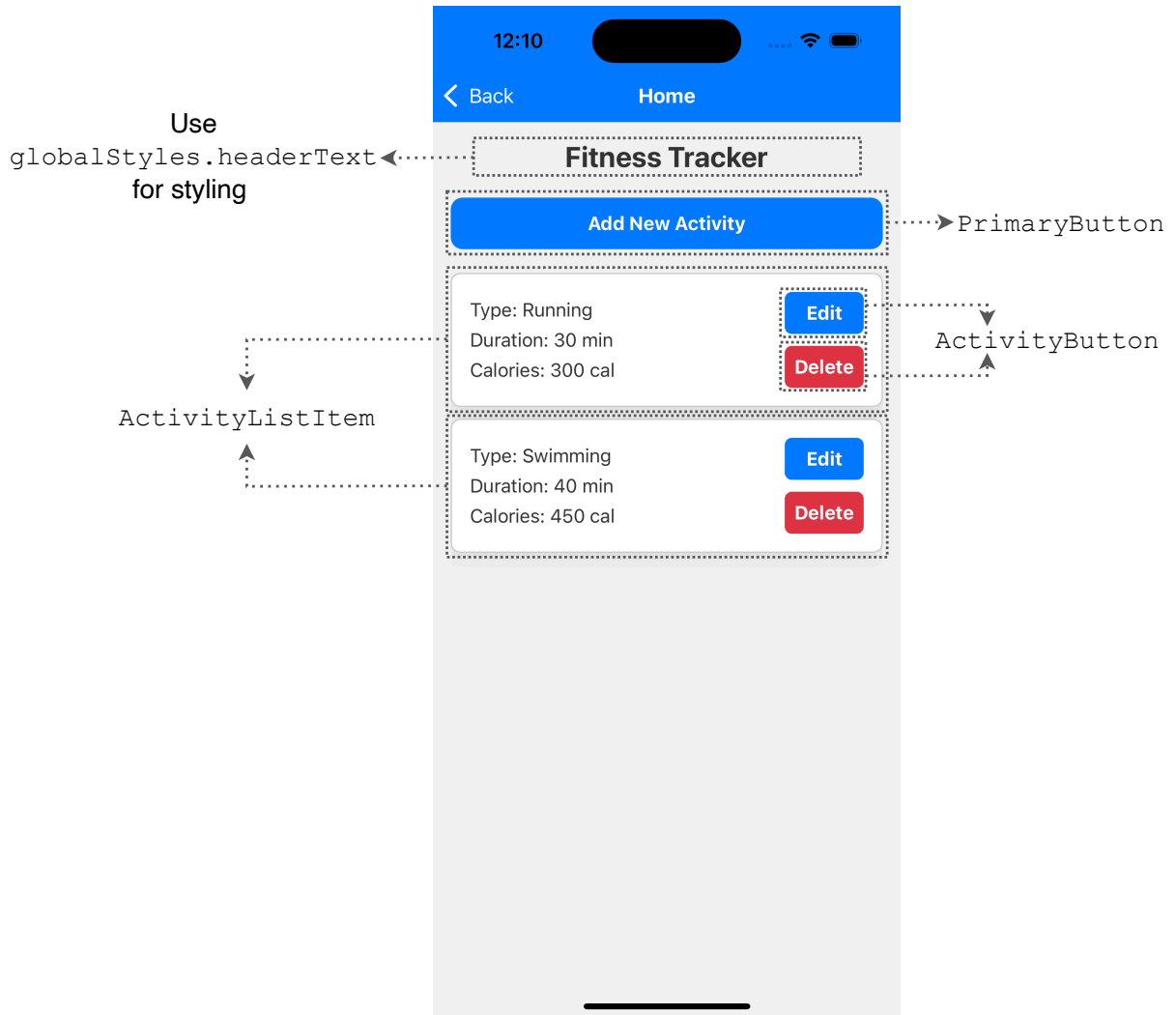
- **Card.tsx (Provided in Starter Code):** A reusable `View` component with card styling (white background, border, rounded corners, padding, shadow/elevation). Use it as a wrapper for activity displays.
 - Example usage: `<Card><Text>Content</Text></Card>`
- **DetailsCard.tsx:** Builds on `Card.tsx` to display activity details (type, duration, calories) in a column layout. Each detail is a `Text` element with styles: `fontSize: 18`, `color: colors.textPrimary`, `marginVertical: 5`.
 - Purpose: Used in the Details screen to show activity info in a clean card format.
 - Props: `{ activity: Activity }`.
 - Example usage: `<DetailsCard activity={activity} />`.
- **ActivityListItem.tsx:** Builds on `Card.tsx` to render a single activity in the list. It includes a tappable area (using `Pressable`) showing details (type, duration, calories) in a column, and a column of Edit/Delete buttons on the right using `ActionButton` with `variant="primary"` for Edit and `variant="danger"` for Delete.
 - Purpose: Used in the Home screen's `FlatList` for each activity item. Tapping the content navigates to Details; buttons handle Edit (navigate to AddEdit with activity) and Delete (remove from state).
 - Props: `{ activity: Activity, setActivities: React.Dispatch<React.SetStateAction<Activity[]>> }`.

- **Styles:** The starter code provides a `StyleSheet.create` with four styles — apply them as follows: `styles.card` to the Card component's style prop; `styles.content` to the `Pressable` (with pressed opacity 0.6); `styles.text` to each detail `Text`; `styles.buttons` to the `View` wrapping the two buttons.
- Example usage: `<ActivityListItem activity={item} setActivities={setActivities} />`.
- **PrimaryButton.tsx:** A styled `Pressable` button for primary actions like “Add New Activity” or form submission.
 - Purpose: Used for prominent actions (e.g., add button on Home, submit on Add/Edit).
 - Props: `{ onPress: () => void, children: string, style?: StyleProp<ViewStyle>, testID?: string | undefined }`.
 - Styles: The starter code provides a `StyleSheet.create` with two styles — apply them as follows: `styles.button` (along with dynamic background color for pressed state) to the `Pressable` style prop; `styles.text` to the `Text` component. Use the pressed parameter in the style function to change the background color of `Pressable` to `colors.primaryPressed` when pressed, otherwise `colors.primary`.
 - Keep `testID` and `accessibilityRole="button"` for autograding (already included in the starter code; do not remove).
 - Example usage: `<PrimaryButton onPress={handleAdd}>Add New Activity</PrimaryButton>`.
- **ActionButton.tsx:** A versatile `Pressable` button with variants for primary (blue) and danger (red) actions.
 - Purpose: Used for secondary actions like Edit (primary variant) and Delete (danger variant) in lists or details.
 - Props: `{ onPress: () => void, children: string, variant: "primary" | "danger", style?: StyleProp<ViewStyle>, testID?: string | undefined }`.
 - Styles: The starter code provides a `StyleSheet.create` with two styles — apply them as follows: `styles.button` (along with dynamic background color for pressed state based on variant) to the `Pressable` style prop; `styles.text` to the `Text` component. Define a `variantColors` object with base and pressed colors for each variant, and use the pressed parameter in the style function to change the background color accordingly.
 - Keep `testID` and `accessibilityRole="button"` for autograding (already included in the starter code; do not remove).
 - Example usage: `<ActionButton variant="danger" onPress={handleDelete}>Delete</ActionButton>`.

3 Screen Implementations

Implement each screen in the `screens/` directory using the custom components, global styles, and Flexbox for layouts. Import `useNavigation` from `@react-navigation/native` for navigation. Pass `setActivities` as a prop where needed for state updates. Use your `NavigationProp` type from `types.ts` to type the `useNavigation` hook for type-safe navigation calls.

1 Home Screen (`screens/HomeScreen.tsx`)



- Displays a header ("Fitness Tracker") using `globalStyles.headerText`.
- A `PrimaryButton` with text `Add New Activity` to navigate to "AddEdit" for adding a new activity.
- A `spacer View` with `height: 20` below the button.
- A `FlatList` rendering `ActivityListItem` for each activity, with `keyExtractor` on `id`, `contentContainerStyle` for bottom padding (`20`), and `testID="activity-list"` for autograding.
- Props: `{ activities: Activity[], setActivities: React.Dispatch<React.SetStateAction<Activity[]>> }`.
- Layout: Wrapped in `globalStyles.container` with `testID="container"`.

2 Add/Edit Screen (`screens/AddEditScreen.tsx`)

For adding a new activity:

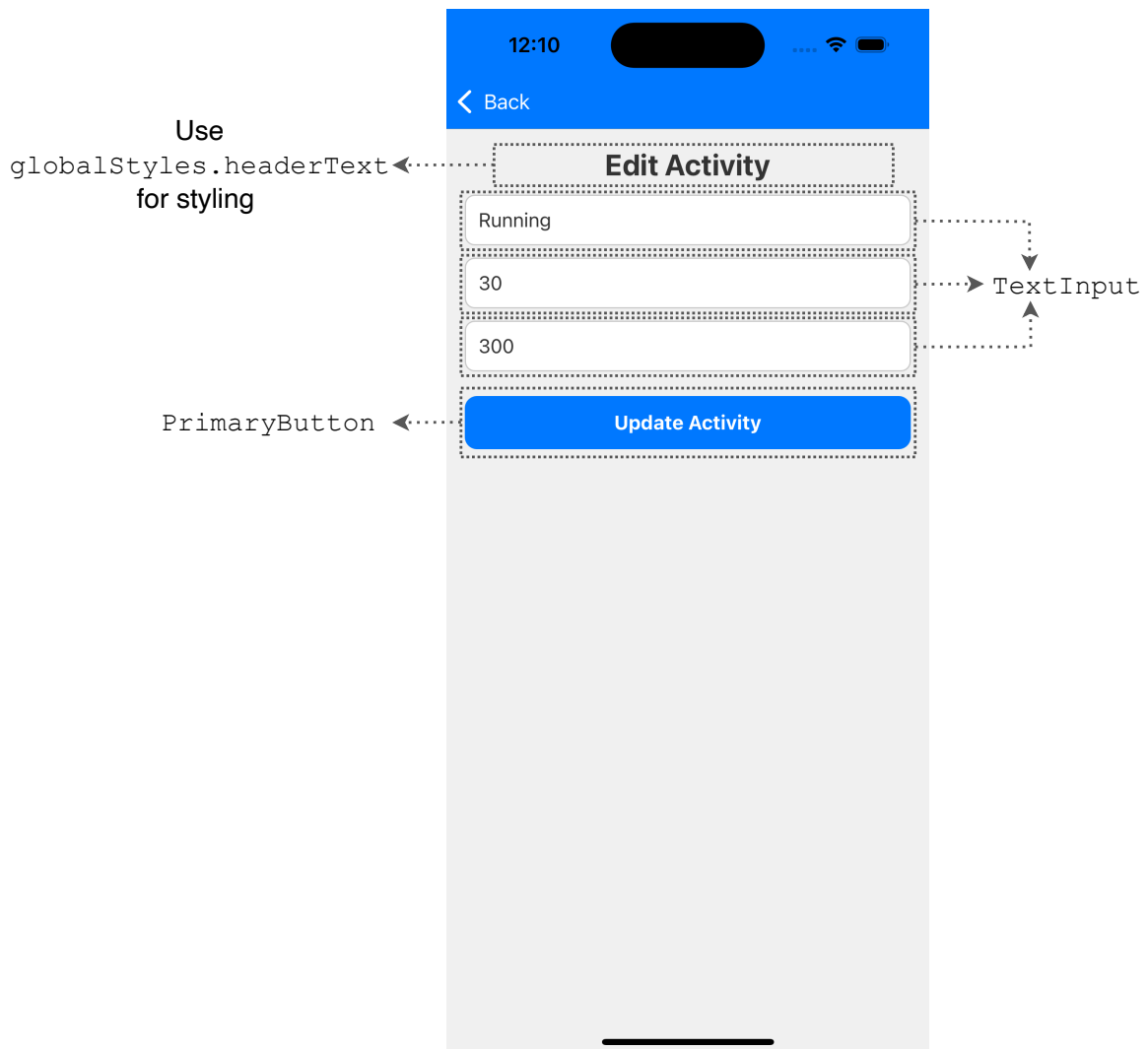
Use
`globalStyles.headerText`
for styling

PrimaryButton

The image shows a mobile app screen titled "Add Activity". At the top is a blue header bar with a white back arrow and the text "Back". Below the header, the title "Add Activity" is displayed in a bold, black font. Underneath the title are three text input fields stacked vertically. The first field is labeled "Activity Type (e.g., Running)", the second is labeled "Duration (minutes)", and the third is labeled "Calories (optional, default: duration * 10)". At the bottom of the form is a large blue button with the text "Add Activity" in white. Annotations with dashed lines point from external text to specific UI elements: "globalStyles.headerText" points to the title, "PrimaryButton" points to the bottom button, and "TextInput" points to the three input fields.

TextInput

For editing an existing activity:

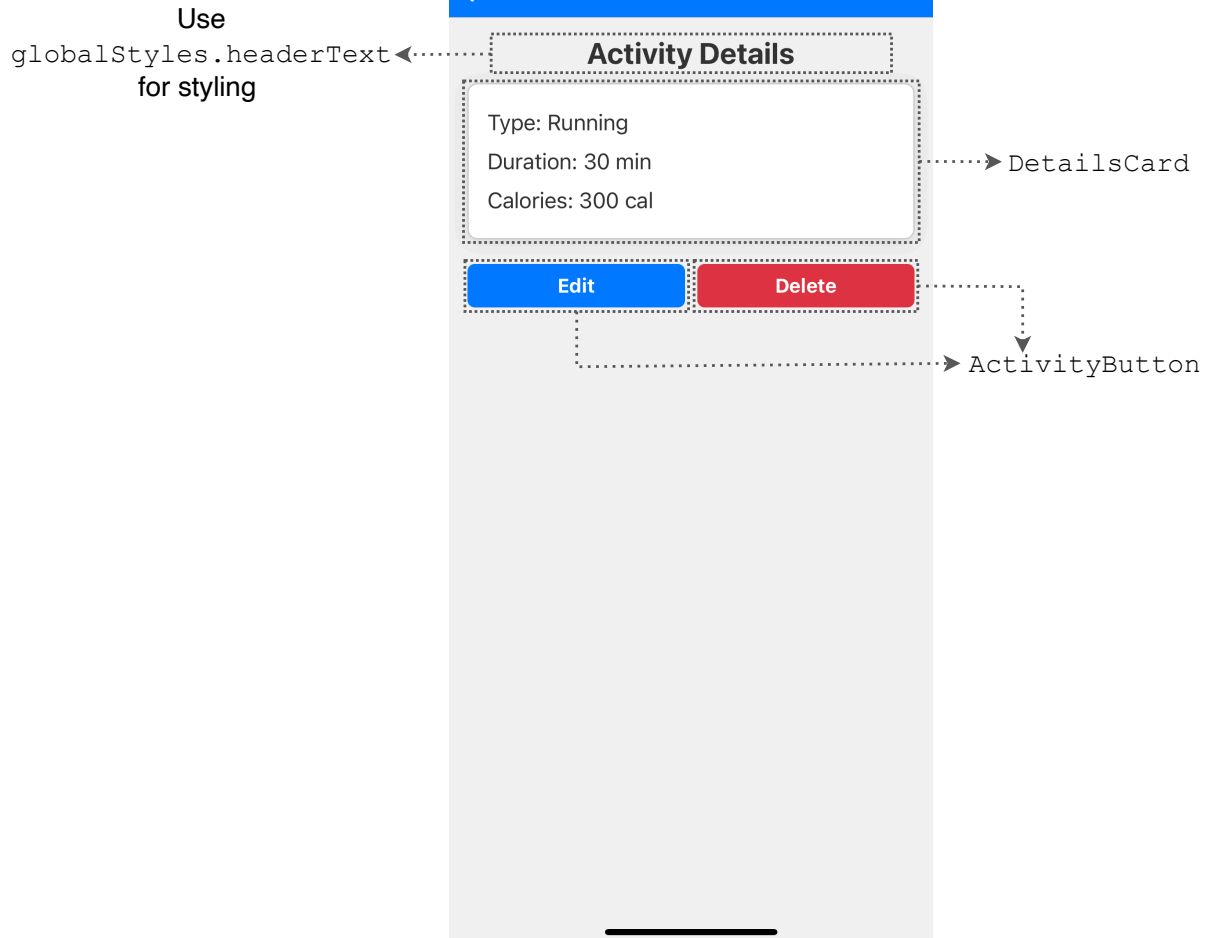


- Header text: "Add Activity" or "Edit Activity" based on whether `route.params.activity` exists.
- Three `TextInput` fields: type (text), duration (numeric), calories (numeric, optional).
- Styles for inputs: Use the provided `input` style for `TextInput`.
- `PrimaryButton` to submit: Validates inputs (type required, duration positive integer, calories optional positive integer or default duration * 10). For adding a new activity, the button text must be `Add Activity` and creates a new activity with a unique id using `Date.now().toString()`. For editing, the button text must be `Update Activity` and updates the existing activity in state. After submission, navigates to "Home" with the updated activity list.
 - Include `testID="add-button"` in this `PrimaryButton` for autograding.
- Use `useEffect` to set navigation title to empty string (avoid redundancy).
- Props: `{ route: { params?: { activity?: Activity } }, setActivities: React.Dispatch<React.SetStateAction<Activity[]>> }`.
- Layout: Column in `globalStyles.container`.

Note

You can directly reuse the input validation logic you implemented in Assignment 1. While the autograder for Assignment 2 will not check input validation (since it is not the focus here), please keep it in your implementation for completeness.

3 Details Screen (`screens/DetailsScreen.tsx`)



- Header: "Activity Details" using `globalStyles.headerText`.
- `DetailsCard` to display the activity.
- Row of `ActionButtons`:
 - Edit: `variant="primary"`, navigates to "AddEdit" with activity, must include `testID="edit-button"`
 - Delete: `variant="danger"`, removes from state and navigates to "Home", must include `testID="delete-button"`
- Styles for buttons: Row with `flexDirection: 'row'`, `justifyContent: 'space-between'`, `marginTop: 10`, `gap: 10`; each button `flex: 1`.

- Props: `{ route: { params: { activity: Activity } }, setActivities: React.Dispatch<React.SetStateAction<Activity[]>> }`.
- Layout: Wrapped in `globalStyles.container`.

4 Multi-Screen Navigation with React Navigation

- In `App.tsx`: Use `useState` for activities. Wrap in `SafeAreaProvider` and `NavigationContainer`.
- Create a `RootStack` component using `createNativeStackNavigator` from `@react-navigation/native-stack`, parameterized with the `RootStackParamList` type you defined in `types.ts` (ensure it reflects the params for each screen: none for "Home", an optional activity for "AddEdit", and a required activity for "Details").
- Screen options: Header background `colors.primary`, tint `colors.white`, title bold/centered, back title "Back".
Note: The back button title is only visible on iOS. On Android, the header shows just the arrow icon by design, which is expected behavior.
- Screens: "Home" passes activities/setActivities; "AddEdit" and "Details" pass setActivities.
- Tappable areas: In `ActivityListItem`, tap content to navigate to "Details" with activity.

Testing Your Implementation

Functionality Checklist

- ☐ Styling applied consistently with `StyleSheet.create` and Flexbox: use `constants/colors.ts` for all colors; apply `styles/globalStyles.ts` for `container` and `headerText` across all screens; ensure card layouts and button styles (primary/danger variants, padding, rounded corners) match requirements.
- ☐ Navigation setup with three screens: Home (list with "Add New Activity" button), Add/Edit (form with conditional "Add Activity"/"Update Activity" button), Details (card with Edit/Delete buttons).
- ☐ Navigate from Home to Add/Edit via "Add New Activity" or Edit button, submit, and return to Home with updated list.
- ☐ Tap activity content in Home list item to view Details; from Details, Edit navigates to Add/Edit (pre-filled), Delete removes and returns to Home.
- ☐ Assignment 1 features (add, update, delete, validation) work across screens with exact error messages preserved.
- ☐ App runs locally via Expo Go or emulator.

Note

The autograder will focus on styling, navigation, and correct screen/component structure. Input validation from Assignment 1 should still be implemented for app completeness, but it will not be re-evaluated.

Manual Testing

1. Start your app locally from your project directory:

```
npx expo start
```

2. Test your app using:

- **Expo Go:** Open the Expo Go app on your iOS/Android device and scan the QR code.
- **Emulator:** Press `i` to launch the iOS simulator, or `a` to launch the Android emulator.
- **Inspector:** Press `m` in the terminal to open the developer menu, then select "Show Element Inspector."

The inspector lets you tap on elements in your app to see their layout boundaries and applied styles. This is especially useful for debugging spacing, alignment, and flexbox issues.

Sample Test Cases

To verify that your app meets the assignment requirements, use the provided test script `a2.sample.test.js`. The autograder will use similar tests to evaluate navigation, styling, and functionality. The test cases in this script correspond to 80% of the assignment grade used by the autograder.

1. [Click](#) to download `a2.sample.test.js`, and save it to your project directory.
2. Execute the test script:

```
npm test
```

Review the output to identify and fix any failing tests. Manually verify navigation and styling in Expo Go or an emulator.

Submission Instructions

1 Generate the Archive

From the parent directory of your project, create a `.tar.gz` archive with:

```
cd /path/to/parent/directory
tar zcvf 1234567890.tar.gz assignment-2
```

- Replace `assignment-2` with the name of your project directory.
- Replace `1234567890` with your student number.
- The archive must contain **exactly one top-level folder** (your project directory) with the required files.
- **Do not** include `node_modules/` or other generated files (e.g., `.expo/`).

Your project directory (e.g., `assignment-2`) should **exactly** match:

- 📁 assignment-2
 - 📄 app.json
 - 📄 App.tsx
 - 📁 assets
 - 📁 components
 - 📄 ActionButton.tsx
 - 📄 ActivityListItem.tsx
 - 📄 Card.tsx
 - 📄 DetailsCard.tsx
 - 📄 PrimaryButton.tsx
 - 📁 constants
 - 📄 colors.ts
 - 📄 index.ts
 - 📄 package-lock.json
 - 📄 package.json
 - 📁 screens
 - 📄 HomeScreen.tsx
 - 📄 AddEditScreen.tsx
 - 📄 DetailsScreen.tsx
 - 📁 styles
 - 📄 globalStyles.ts
 - 📄 tsconfig.json
 - 📄 types.ts

⚠️ Any submission that fails to meet the specified format or structure and requires TA intervention for the autograder to work will receive a **20-point deduction**.

2 Verify Submission Structure

To avoid a 20-point deduction, use the provided Python script to check your `.tar.gz` file before submission.

1. [Download](#) `check_submission_structure.py` and save it in the parent directory of your project.
2. Ensure you have Python installed. Check by running `python --version` in your terminal/command prompt. If not installed, download it from python.org.
3. From the parent directory of your project, run:

```
python check_submission_structure.py 1234567890.tar.gz
```

- Replace `1234567890` with your student number.

3 Submit to Quercus

Submit your `.tar.gz` file to [Quercus](#).

Note

You are allowed to submit unlimited times. Only your latest submission before the deadline will be graded. Quercus automatically appends a suffix to the file name after the first submission — this will not affect grading.

Grading Scheme (100 Points)

The autograder runs the app via `npx expo start`, testing navigation, styling, and component usage. Partial marks are awarded for partial functionality. Assignment 1 validation is not retested, but core add/edit/delete functionality must work.

App Setup and Navigation (7 Points)

- Stack navigator screen options (2 pts, 0.4 each).
- `types.ts` defines `RootStackParamList` (3 pts), `NavigationProp` (1 pt), `Activity` retained (1 pt).

Home Screen (30 Points)

- Structure: `PrimaryButton` "Add New Activity" navigates to `AddEdit` (4 pts), spacer `View` (height 20, 2 pts), `FlatList` with correct props (4 pts).
- Styling: `globalStyles.headerText` for "Fitness Tracker" (7 pts), `globalStyles.container` (3 pts), `ActivityListItem` uses `Card` with `styles.card` (4 pts), `Pressable` with `styles.content /opacity` (2 pts), `Text` with `styles.text` (4 pts).

Add/Edit Screen (14 Points)

- Structure: Conditional header "Add Activity"/"Edit Activity" (3 pts), `PrimaryButton` "Add Activity"/"Update Activity" adds/edits and navigates to Home (6 pts).
- Styling: `TextInput` with provided styles (2 pts), `PrimaryButton` with provided styles and pressed state (3 pts).

Details Screen (19 Points)

- Structure: Header "Activity Details" (2 pts), `ActionButton` s: Edit navigates to `AddEdit` (3 pts), Delete removes/navigates to Home (4 pts).
- Styling: `DetailsCard` with `Card` and activity data (4 pts), `Text` styles (4 pts), buttons View with `Flexbox` (2 pts).

Navigation Flows (30 Points)

- Home → Add/Edit → Home (7 pts)

- (3 pts) "Add New Activity" button correctly navigates to the Add/Edit screen.
- (2 pts) Submitting adds a new activity with correct type, duration, and calculated calories.
- (2 pts) Returning to Home shows the updated list with the new activity.
- **Home → Details (7 pts)**
 - (4 pts) Tapping an activity opens the Details screen showing the correct activity data.
 - (3 pts) The Details screen displays type, duration, and calories correctly.
- **Details → Edit → Home (8 pts)**
 - (3 pts) "Edit" button navigates to Add/Edit with the selected activity loaded.
 - (3 pts) Submitting updates the activity correctly in state.
 - (2 pts) Returning to Home shows the updated list.
- **Details → Delete → Home (8 pts)**
 - (3 pts) "Delete" removes the activity from the list.
 - (2 pts) Navigates back to Home after deletion.
 - (3 pts) The deleted activity no longer appears in the list.

Resources

- **Course Materials:**
 - [Lecture 3: Styling in React Native](#)
 - [Lecture 4: Navigation with React Navigation](#)
- **External Resources:**
 - [React Native Styling](#) ↗
 - [Flexbox in React Native](#) ↗
 - [React Navigation Documentation](#) ↗
 - [Expo with React Navigation](#) ↗

Questions?

1. Discussion Board:

- Post questions on [course discussion board](#) ↗
- Search existing discussions first
- Use clear titles and provide relevant code snippets

2. Office Hours:

- Time: Mondays, 2:00 PM — 3:00 PM
- Location: Room 7206, Bahen Centre for Information Technology

3. Tips for Getting Help:

- Start early to allow time for questions
- Be specific about your problem
- Share what you've tried
- Include relevant error messages

Last updated on October 18, 2025